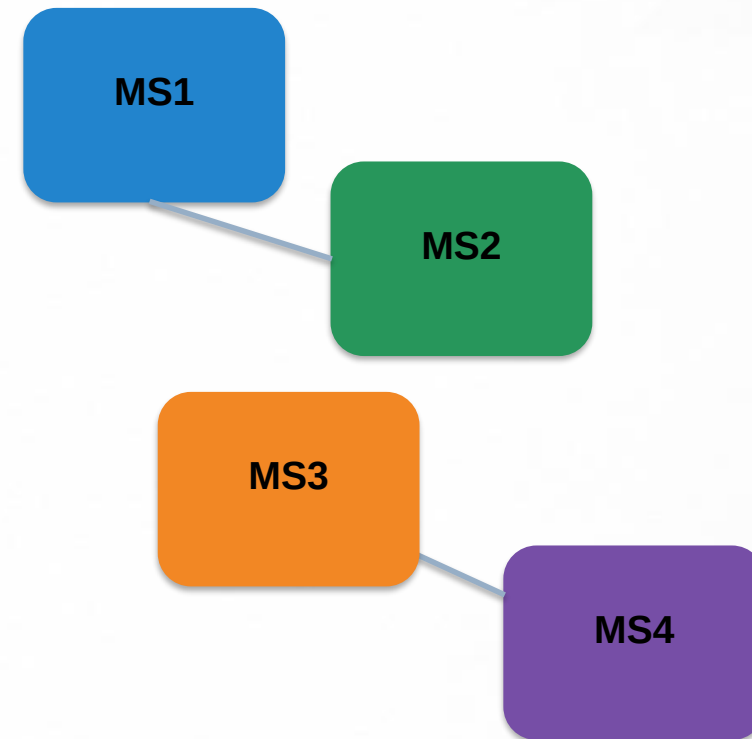


SBSE no contexto de microserviços

Hélio Toshio Kamakawa · Kananda Caroline

Profa. Thelma Elita Colanzi
Prof. João Choma Neto



Contexto e motivação

- Sistemas nascem monolíticos → simples no início, mas acumulam acoplamento e ficam difíceis de manter e evoluir.
 - **Migração monolito → microsserviços:** a etapa mais crítica é decidir *onde cortar* - quais classes, módulos e tabelas ficam juntos e quais se separam. É a decisão que guia toda a apresentação.
 - **Por que é difícil (1):** critérios que competem entre si coesão × acoplamento × granularidade. Melhorar um piora o outro; não há resposta única.
 - **Por que é difícil (2):** o nº de partições cresce combinatorialmente (números de Bell). Poucas dezenas de classes → número astronômico → busca exaustiva inviável.
 - **Abordagem:** tratar como otimização → **SBSE** (Search-Based Software Engineering)

O que é SBSE?

Espaço de Busca

Conjunto de todas as soluções possíveis. Cada solução candidata = uma decomposição completa do sistema (uma forma de dividir as classes em microsserviços). Representada por um vetor de inteiros: cada posição = uma classe, o valor = a qual microsserviço ela pertence.

Função objetivo (fitness)

Como a gente mede se uma solução é boa ou ruim. As métricas: coesão (o quanto as classes de um mesmo microsserviço se relacionam), acoplamento (o quanto microsserviços diferentes dependem uns dos outros) e granularidade (tamanho e número de microsserviços). É o que o algoritmo usa pra comparar uma decomposição com outra.

Algoritmo de busca

O mecanismo que vasculha o espaço enorme de soluções, guiado pela função objetivo, procurando as melhores decomposições. Como o espaço é gigante (números de Bell), ele explora de forma inteligente, sem testar tudo.

Exemplo de ESPAÇO DE BUSCA

Vetor de decomposição: cada posição é uma classe, o valor é o microserviço

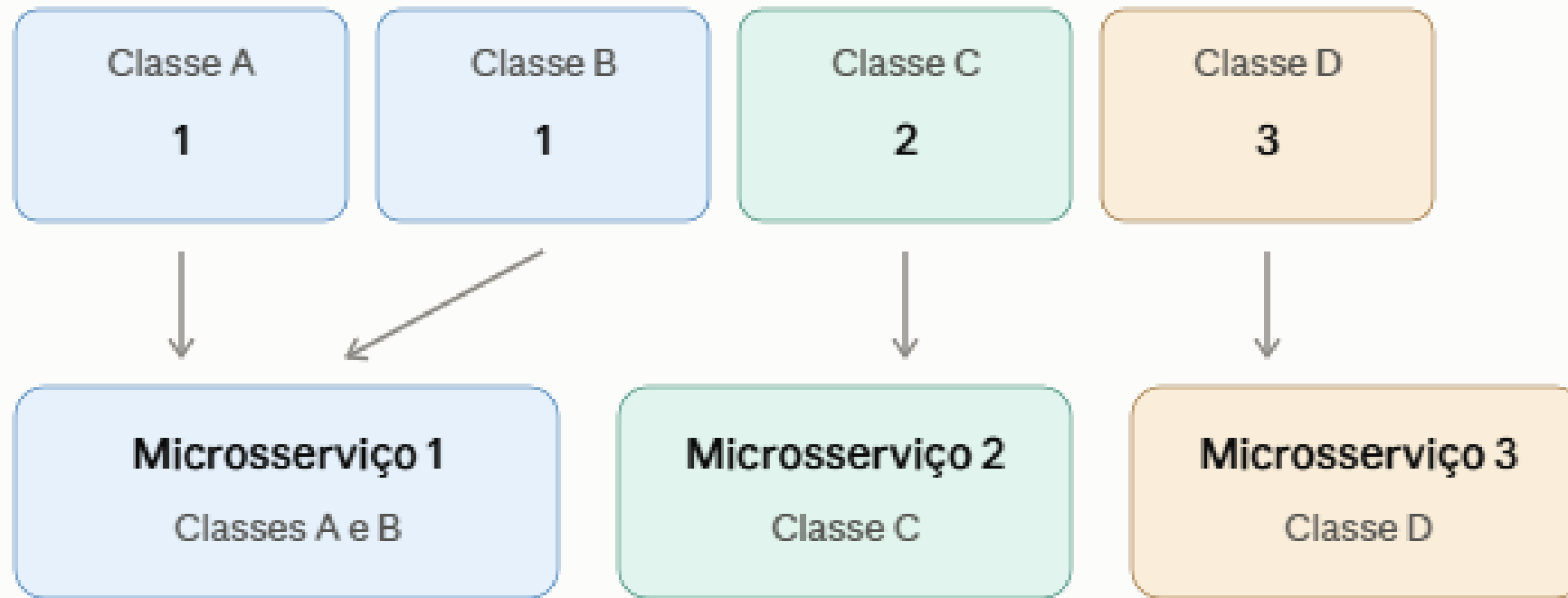
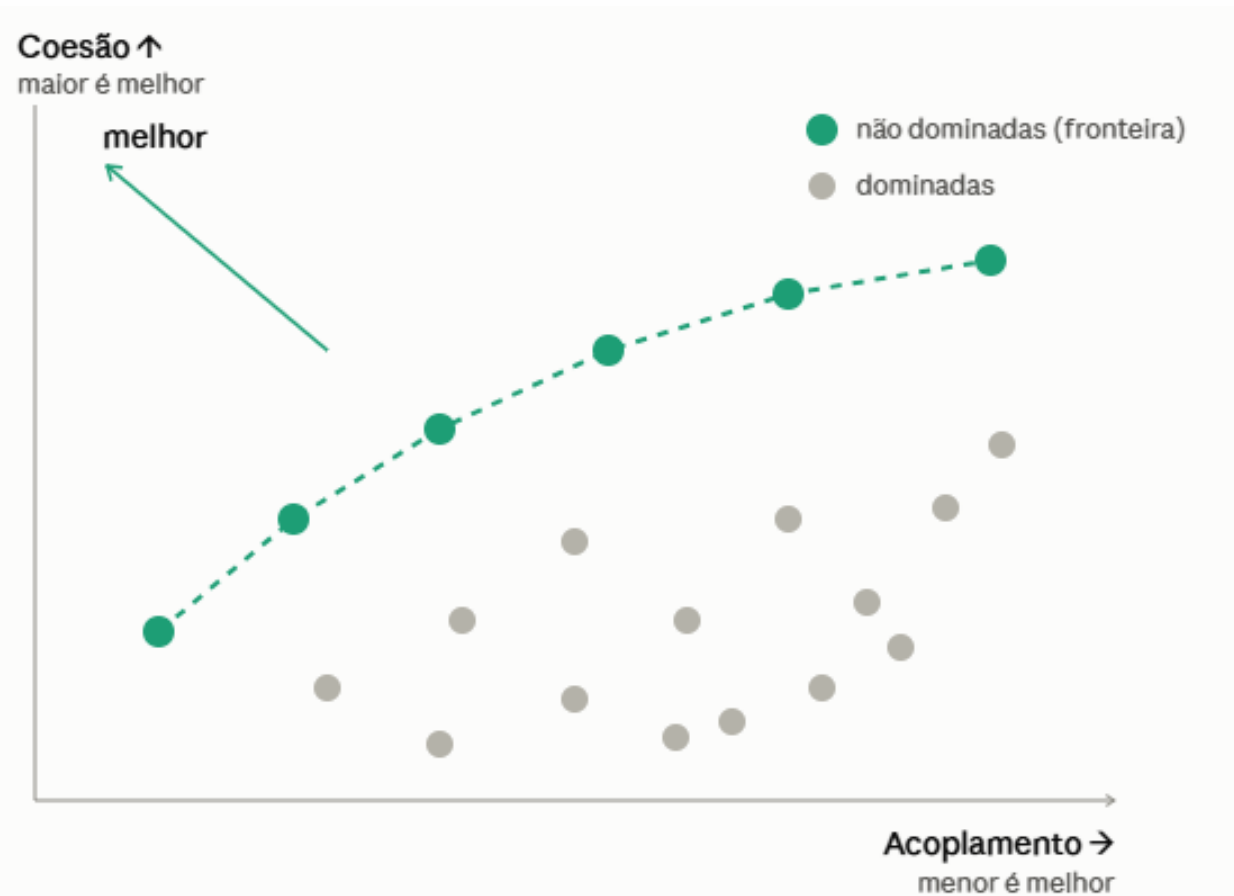


Figura 2. Exemplo de Fronteira de Pareto considerando coesão e acoplamento.
Fonte: elaboração própria.

Objetivos conflitantes e Fronteira de Pareto

- **Coesão × acoplamento competem:** coesão quer *juntar* classes relacionadas; acoplamento quer *separar* (menos dependência). Melhorar um piora o outro.
- **Sem solução única:** não dá pra otimizar os dois ao extremo → usam-se **MOEAs** (algoritmos evolutivos multiobjetivo), que não devolvem uma resposta só.
- **Fronteira de Pareto:** conjunto de soluções *não dominadas* - nenhuma vence a outra em tudo; cada uma é um trade-off legítimo.
- **O algoritmo não escolhe por você:** ele entrega o leque de boas opções → o arquiteto escolhe a adequada ao projeto.
- Ex.: **NSGA-II · IBEA · MOEA/D** (aparecem nos três trabalhos)

Fronteira de Pareto: coesão × acoplamento



Monólito × Microserviços

Monólito

- Construído, implantado e executado como uma única unidade
- Todos os módulos no mesmo processo; um banco de dados compartilhado
- Simples no início (desenvolvimento, deploy, testes)
- Cresce → acoplamento interno forte → difícil manter, evoluir e escalar

Microserviços

- Serviços menores e independentes, cada um com uma capacidade de negócio
- Dados próprios (database-per-service) — evita compartilhar BD
- Escalabilidade granular, autonomia de equipes, isolamento de falhas
- Preço: desafios de sistema distribuído — rede, latência, consistência

Banco de dados como dimensão de acoplamento

Database-per-service

Cada microserviço gerencia o próprio banco, inacessível aos outros. Esse isolamento é o que garante o desacoplamento em tempo de execução permite evoluir, implantar e escalar de forma independente. Se violado (duas tabelas de serviços diferentes no mesmo esquema), o acoplamento de dados subverte a decomposição, mesmo com o código bem separado.

Transações distribuídas

No monólito, uma operação era uma transação ACID local ou tudo acontece, ou nada, com garantia. Separando os dados em bancos independentes, vira uma sequência de operações sobre vários bancos, sem essas garantias. Particionar sem olhar o esquema pode criar, sem querer, uma dependência transacional entre serviços que deveriam ser independentes.

Chaves estrangeiras cruzando fronteiras

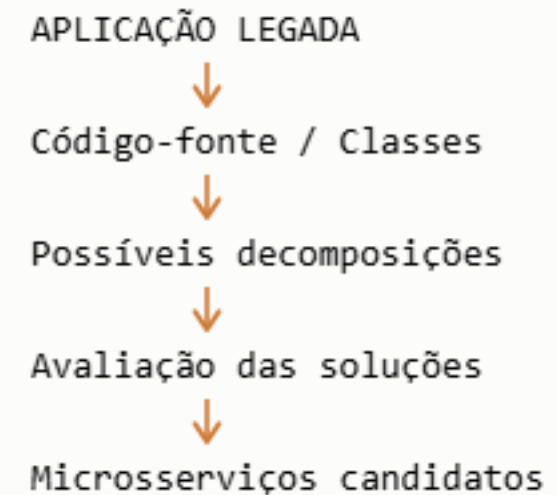
Uma FK que cruza fronteiras de domínio é um sinal DIRETO e OBJETIVAMENTE MENSURÁVEL de acoplamento de dados. Por quê? Duas tabelas ligadas por FK dificilmente vão pra microserviços diferentes sem chamadas de rede síncronas ou duplicação de dados pra manter a integridade. Ponto-chave: esse sinal está no próprio esquema físico do banco legado → calculável ESTATICAMENTE, antes de rodar o sistema.

O problema de identificação de fronteiras

- Particionar os elementos do sistema legado (classes, módulos, tabelas) em serviços candidatos é a decisão que discutimos desde o começo.
- Não é encontrar, é construir: os microsserviços não existem escondidos no monólito. Você cria a divisão do zero → é um problema de projeto, não de descoberta.
- Por que a força bruta não funciona: o nº de partições cresce combinatorialmente (números de Bell) → testar todas é inviável → justifica o SBSE.
- O tamanho do problema: $1 \cdot 2 \cdot 5 \cdot 15 \cdot 52 \cdot 203 \cdot 877 \cdot 4.140 \cdot 21.147 \cdot 115.975 \dots$
- 10 classes → +115 mil partições · 20 classes → +51 trilhões · sistemas reais têm centenas (Roller = 534)

MSExtractor: Proposta

- **A PROPOSTA:**
- Identificar automaticamente POSSÍVEIS fronteiras de microsserviços em aplicações legadas orientadas a objetos.
- Repare em "possíveis fronteiras" não dá resposta única, sugere candidatas, como vimos na Fronteira de Pareto.



Sellami et al. (2022) - MSExtractor

- Publicado em *Information and Software Technology* (Elsevier, Q1)
- Formula a extração como **otimização combinatória multiobjetivo** (algoritmo **IBEA**)
- Papel do banco de dados: **ausente** só dependências de código

Coesão

- A coesão de um microserviço m é calculada como o complemento da média de CS entre todos os pares de classes pertencentes a m .
- A coesão da decomposição M agrega esse valor para todos os microserviços:

$$Cohesion(\mathcal{M}) = 1 - \frac{\sum_{m \in \mathcal{M}} Coh(m)}{|\mathcal{M}|}, \quad \text{a ser maximizada}$$

Acoplamento

- O acoplamento entre dois microserviços m_1 e m_2 é calculado exclusivamente sobre suas classes de interface (I_1 e I_2), pois são elas que expõem funcionalidades para outros serviços.
- As classes inner são excluídas por não participarem da comunicação entre microserviços:

$$Granularity(\mathcal{M}) = \frac{|C|}{|\mathcal{M}|} \cdot \sum S(c_i, c_j)$$
$$C(m_1, m_2) = \frac{\sum_{c_i \in I_1, c_j \in I_2} S(c_i, c_j)}{|I_1| \times |I_2|}$$

- **Granularidade.** A granularidade é a razão entre o número total de classes $|C|$ e o número de microserviços $|\mathcal{M}|$:

$$Granularity(\mathcal{M}) = \frac{|C|}{|\mathcal{M}|}$$

Formulação SBSE

- Formula a extração de microsserviços de sistemas legados OO como **otimização combinatória multiobjetivo**, resolvida pelo algoritmo **IBEA**.
- **Representação:** vetor de inteiros (posição = classe, valor = microsserviço). Distingue classes *inner* (uso interno) de *interface* (expostas) → só as de interface contam no acoplamento.
- **Objetivos:** coesão · acoplamento · granularidade calculados **só de dependências de código** (o BD não entra).
- **Algoritmo:** IBEA, comparado a NSGA-II e SPEA2 → provam que o IBEA é a melhor escolha.

Avaliação

- **Parte 1 comparação com concorrentes:** 4 sistemas legados open-source (JPetStore, SpringBlog, JForum, Roller de 24 a 534 classes). Comparado com FoME, MEM e LIMBO.
- **Parte 2 comparação com *ground truth*:** 3 projetos que já nasceram em microsserviços. Junta tudo num monólito, roda o MSExtractor e mede o quanto ele reconstrói a divisão original (precisão / revocação).
- **Sem participantes humanos:** validação 100% automática.

Resultados

- **Supera os concorrentes:** vence FoME, MEM e LIMBO na maioria das métricas de coesão e acoplamento. Vantagem **maior nos sistemas grandes** (nos pequenos, métodos simples já bastam). Ex. Roller: 946 interações vs 1441 (FoME) vs 2786 (MEM) menor é melhor.
- **IBEA é o melhor algoritmo:** supera NSGA-II e SPEA2 nos 4 sistemas, com significância estatística (Wilcoxon $p < 0,05$) e efeito grande. Eles *provam*, não só afirmam.
- **Vs. decomposição humana (*ground truth*):** precisão **0,73–0,91** (alta) e revocação **0,61–0,74** (mais baixa).
- **Por que a revocação cai:** erra ao agrupar classes quase idênticas e isso acontece por causa do banco de dados.
- **Papel do BD: ausente.** Objetivos 100% de código. O banco aparece no artigo só como *causa de erro*, nunca como critério de decisão.

Publicação

Onde os estudos foram publicados?

SAMIM

- **FRONTIERS OF COMPUTER SCIENCE**
- Q1 · IF $\approx 7,3$ (JCR)
- **ESCOPO**
- Computação / Engenharia de Software

MSDC

- **IEEE ACCESS**
- Q2 IF $\approx 3,4$ · acesso aberto
- **ESCOPO**
- Multidisciplinar

Jornal – Revisão por pares às cegas

Autores

Formação, vínculo institucional e experiência



SAMIM

- Sedigheh Khoshnevis
- **CONTEXTO ACADEMIA**
- Professora assistente · Islamic Azad University
- **FORMAÇÃO / EXPERIÊNCIA**
- PhD em Eng. de Software (2015)



MSDC

- Kinoshita & Kanuka
- **CONTEXTO INDÚSTRIA**
- Departamento P&D · Hitachi · Japão
- **FORMAÇÃO / EXPERIÊNCIA**
- Kinoshita · Mestre em Engenharia · Pesquisador
- Kanuka · Mestre em Engenharia · Pesquisador-chefe



SAMIM

Identificar MS considerando diferentes
necessidades dos clientes de SaaS

SAMIM - Processo

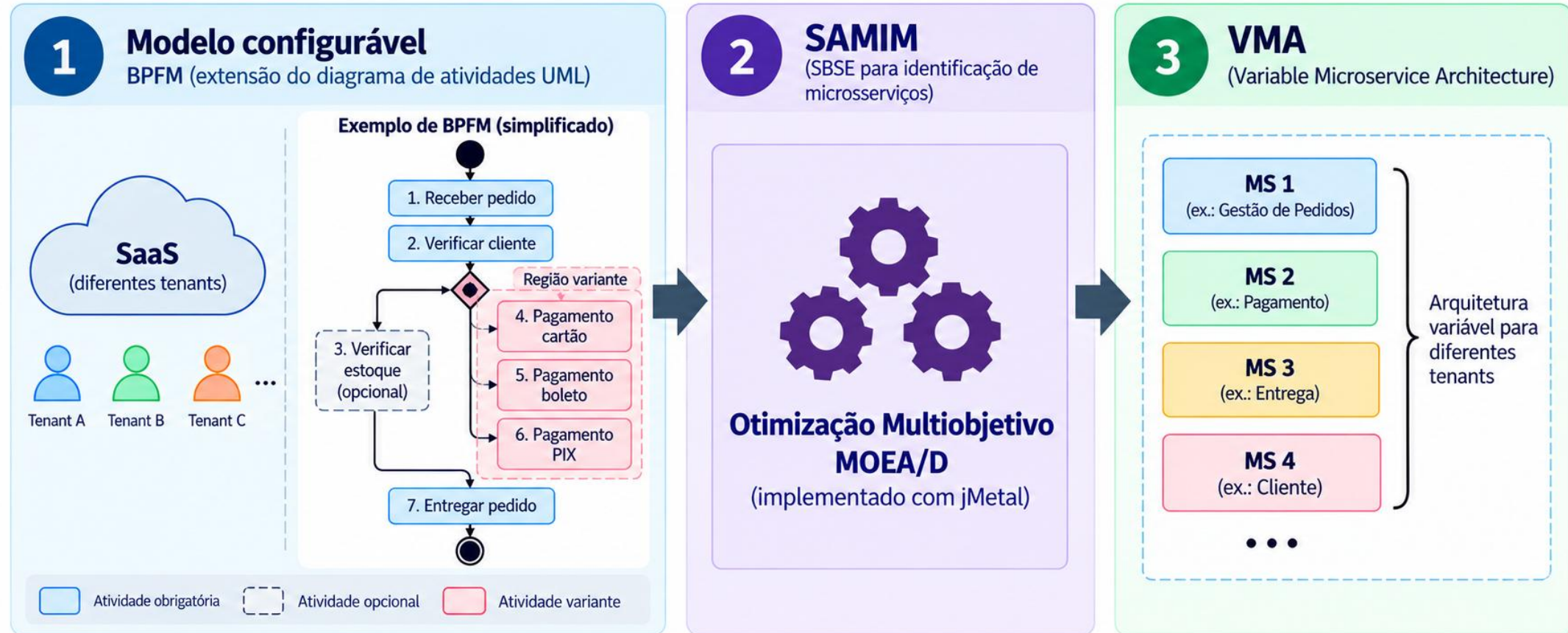
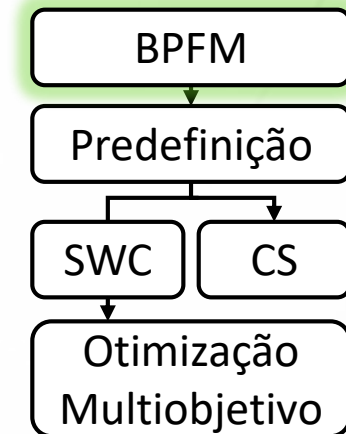


Figura 3. Processo geral do SAMIM. Fonte: adaptado de Khoshnevis [2]

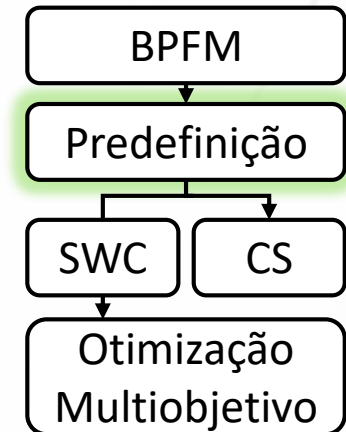
Entrada (BPFM)

- BPFM – Modelo de Família de Processo de Negócio
 - Notação de modelagem baseada em Diagrama de Atividades UML
 - Atividades obrigatórias/opcionais
 - Pontos de variação;
 - Atividades variantes;
 - Regiões variantes
 - Resultado da modelagem (ENTRADA SAMIM): REPRESENTAÇÃO estruturada do modelo BPFM
 - Experimento:
 - **BPFM foi fornecida para todos avaliadores (proveniente de 3 empresas de manufatura de venda)**
 - 27 avaliadores identificaram manualmente os VMA



Predefinição

- Prepara a entrada para a otimização
- Gera 2 representações:
 - SWC — Single Weighted Configuration
Configuração Única Ponderada
 - CS — Configuration Set
Conjunto de Configurações



Representações

- Configuração Única Ponderada (SWC)

- **Resume a variabilidade por meio de pesos**

- Conta ocorrências das atividades em configurações válidas.

- Exemplo: A1 = Atividade pedido;
7 ocorrências em 10 configurações

- $7/10 \rightarrow A1 = 0,7$

- $SWC = \{(A1, 1.0), (A2, 1.0), (A3, 0.5), \dots\}$

- Conjunto de Configurações (CS)

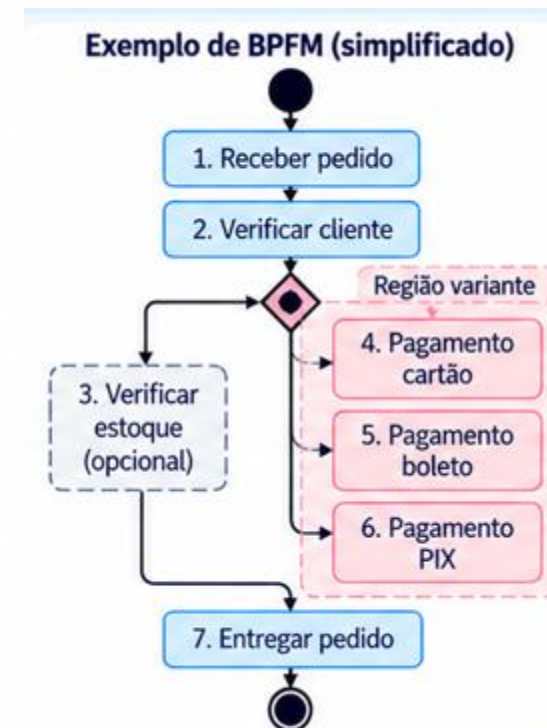
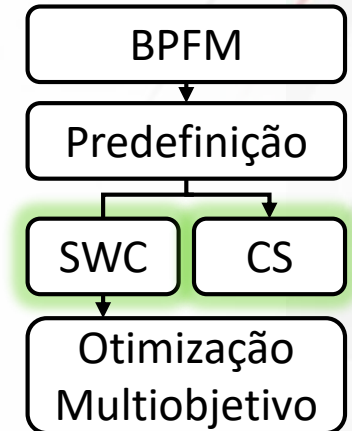
- **Preserva as configurações válidas do BPFM**

- $A \rightarrow B \rightarrow \text{Variação (C,D,E)} \rightarrow F$

- Configuração 1: $A \rightarrow B \rightarrow C \rightarrow F$

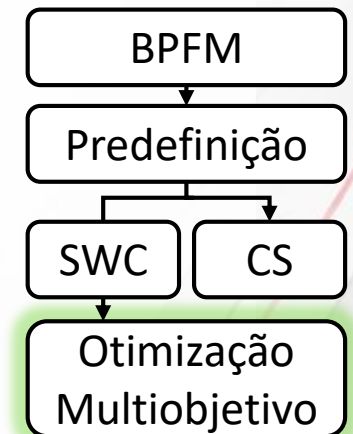
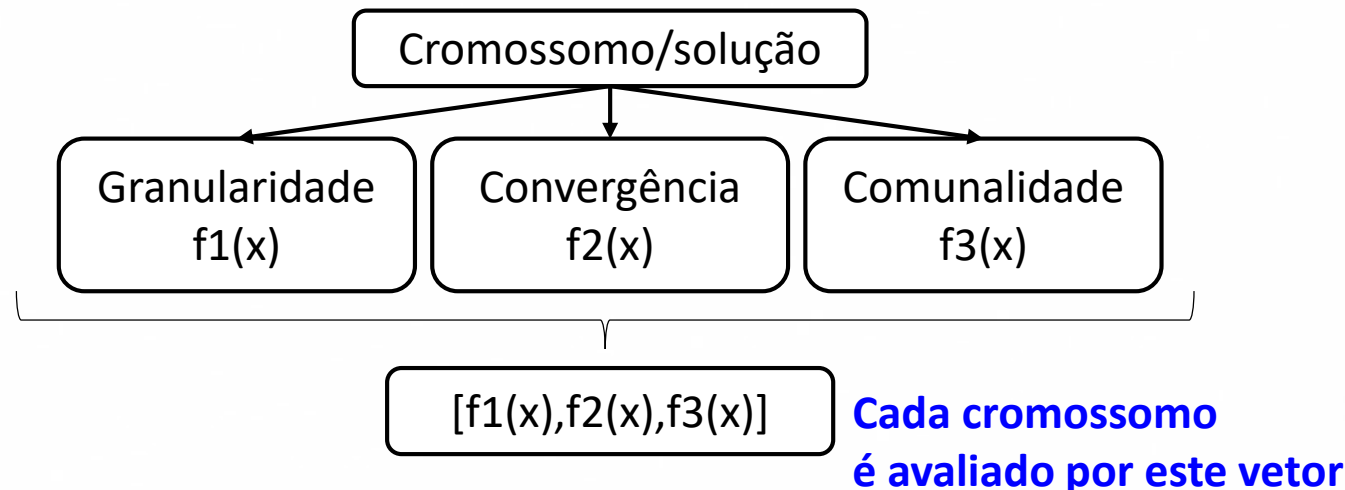
- Configuração 2: $A \rightarrow B \rightarrow D \rightarrow F$

- Configuração 3: $A \rightarrow B \rightarrow E \rightarrow F$



Otimização – MOEA/D

- Algoritmo Evolutivo Multiobjetivo baseado em Decomposição
 - Trabalha com uma população de soluções (cromossomo)
 - Multiobjetivo (**convergência, granularidade e comunalidade**)
 - D – baseado em decomposição
 - Em vez de tratar diretamente os **3 objetivos** em conjunto, decompõe o problema multiobjetivo em problemas menores, cada um representando uma forma de combinar/priorizar os objetivos



Otimização

- Cromossomo é representado por um vetor

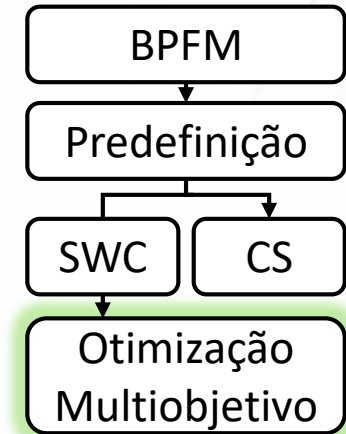
Posição: atividade

Atividade	A	B	C	D
MS	1	1	2	2

Valor: MS

[1,1,2,2]

Isso significa:
MS1{A,B} e MS2{C,D}



- Os cromossomos são gerados aleatoriamente na população inicial

Otimização - gerações

- População inicial
 - Busca diferentes agrupamentos de atividades em MS
 - 10.000 gerações - 500 soluções na população
- Produz novas gerações:
 - Seleção
 - Torneio binário – duas soluções são colocadas em competição
 - Cruzamento
 - Evolução diferencial - utiliza diferenças entre soluções para gerar novas candidatas
 - Taxa de cruzamento: 0,95 (95%)
 - Mutação
 - Mutação polinomial - introduz alterações na solução, favorecendo pequenas mudanças
 - Probabilidade: 0,01 (1%)
- **Durante a evolução, cada cromossomo é avaliado nos 3 objetivos**

Convergência de Entidades de Negócio

- Avalia uma solução candidata de acordo com a **MATRIZ Entidade-Atividade**.
 - Considera leitura (r), escrita (w) ou ambas (r/w);
 - A matriz EA atribui valores: leitura (r) = 0,5; escrita (w) = 1; leitura/escrita (r/w) = 1,5
- O SAMIM busca maximizar a convergência

$$\sum_{i=1}^a \left(p(a_i) * \sum_{j=1}^b ea_{ij} \right) \left(\frac{1}{ms} + \frac{1}{b} \right)$$

nº entidades negócio

“Pagamento em dinheiro” ESCRIVE uma entidade

$$p(A1) * w \Rightarrow 0,5 * 1,0 = 0,5$$

Pagamento

Escrita (1,0)

Granularidade

- Busca granularidade **intermediária**, chamado **GG**.

Exemplo 12 atividades

$$\frac{\text{atividades}}{\text{n}^{\circ} \text{ ms}} \quad \frac{12}{3} = 4 \quad \frac{12}{12} = 1 \quad \frac{12}{1} = 12$$

$$GG = \frac{\frac{12}{1} + \frac{12}{2} + \dots + \frac{12}{12}}{12} \therefore \frac{\cancel{12}(\frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{12})}{\cancel{12}} \therefore GG \approx 3,10$$

Grau de Comunalidade

- Buscar cromossomos balanceadas com MS comuns e variáveis;
- MS comum é o que tem atividade obrigatória;
- O estudo argumenta que um grau de comunalidade muito **alto** ou muito **baixo** não é desejável e estabelece **0,5** como **valor-alvo**.

$$VCOMM(x) = \frac{cms}{ms}$$

Caso 1 – COMM alta
MS1{A,C}, MS2{B,D},
MS3{F,E} $\frac{3}{3} = 1$

Caso 2 – COMM baixa
MS1{A,B,F}, MS2{C},
MS3{D,E} $\frac{1}{3} = 0,33$

Caso 3 – alvo
MS1{A,B}, MS2{C,D},
MS3{E}, MS4{F} $\frac{2}{4} = 0,5$

Vetor Objetivo

- Os valores dos 3 objetivos formam o Vetor de Objetivos:

$$F(x) = [f_1(x), f_2(x), f_3(x)]$$

- Utilizado para posicionar a solução no **espaço de objetivos**.
- Cada cromossomo x possui seu próprio vetor objetivo.
- **Com os cromossomos avaliados e suas posições no espaço de objetivos definidas, seguimos para a seleção.**

Seleção Final

- O que existe depois das 10.000 gerações?
- **Conjunto de Pareto**
 - Mantém soluções **não dominadas**
 - Ainda assim, há muitas soluções
- **Tchebycheff (seleciona uma única final)**
 - Escolhe uma solução do conjunto de Pareto
 - Pesos iguais para os 3 objetivos
 - Ponto ideal: [0, 0, 0]
 - Busca minimizar o **pior resultado** entre os objetivos

$$A=[0,2;0,1;0,3]$$

$$B=[0,7; 0,4; 0,1]$$

$$\max(A)=0,3 < \max(B)=0,7$$

A escolhida

VMA - Arquitetura de MS Variabilidade

- Neste fase o conjunto de configurações (CS) é utilizada para:
 - Classificar MS em obrigatórios ou opcionais
 - Identificar relações includes / excludes entre os opcionais
- Saída: Arquitetura Variável de Microsserviços (VMA)

Experimento – objeto de avaliação

- O que foi utilizado para avaliar cada método?

6 casos experimentais

- (1) Processamento de Pedido de Compra
- (2) Processamento de Compra
- (3) Processamento de Ordem de Serviço
- (4) Compras Online
- (5) Submissão de Documentos
- (6) Lançamentos Contábeis

27 participantes

- **SELEÇÃO**
 - 556 contatados
 - 27 qualificados
- **CRITÉRIOS**
 - Arquitetura e projeto de software
SaaS · microsserviços · SOA · variabilidade
- **PERFIL**
 - Especialistas da indústria
 - Estudantes de graduação/pós-graduação
 - Professores de Engenharia de Software

Experimento - Método de avaliação

- Comparação baseline (escalar e estrutural)
 - Especialistas realizaram a identificação dos MS;
 - As decomposições humanas foram comparadas à solução gerada pelo SAMIM;
 - Avaliação baseada no grau de similaridade entre as decomposições;

Resultados

- Quais foram os resultados da avaliação?

71,57% similaridade escalar (média)

Em alguns casos, resultados até

57,85% mais próximos da meta que a média humana

42,95% similaridade estrutural (média)

Comparação escalar da convergência entre entidade atividade

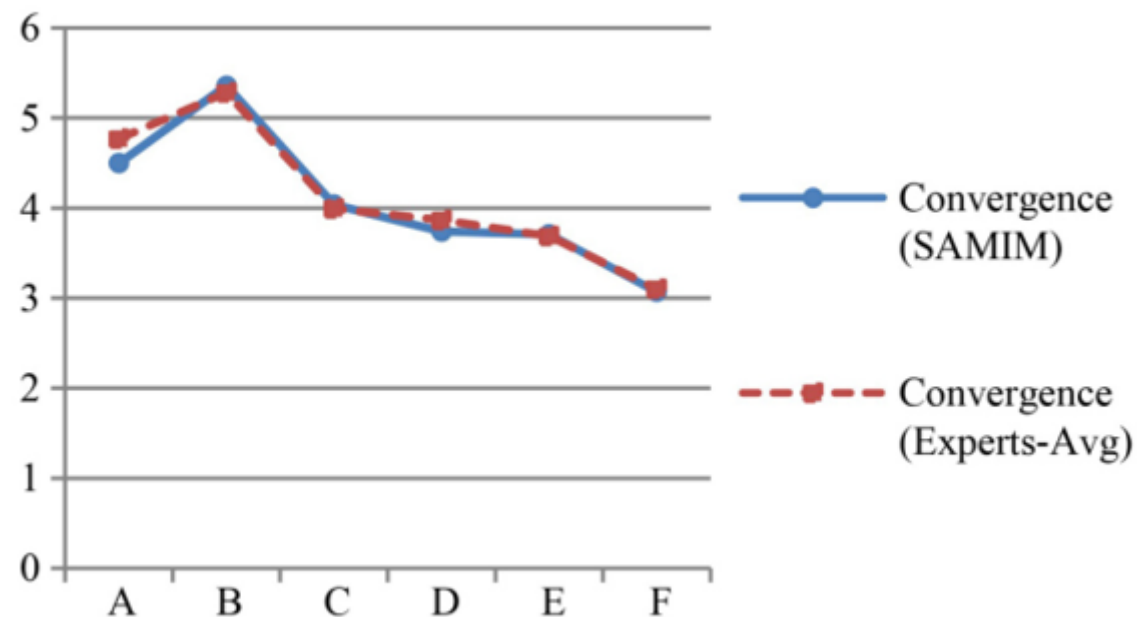


Figura 4. Comparação da convergência entre entidades de negócio e atividades para os seis casos experimentais. Fonte: Khoshnevis [2]

Comparação escalar granularidade e Comunalidade

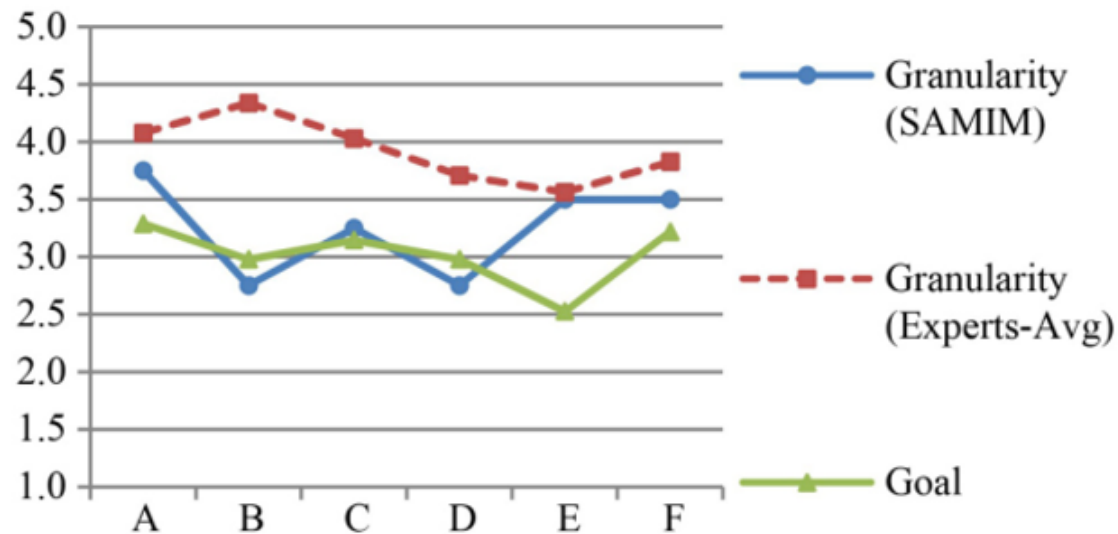


Figura 5. Comparação dos valores de granularidade obtidos pelo SAMIM e pelos especialistas nos seis casos experimentais. Fonte: Khoshnevis [2].

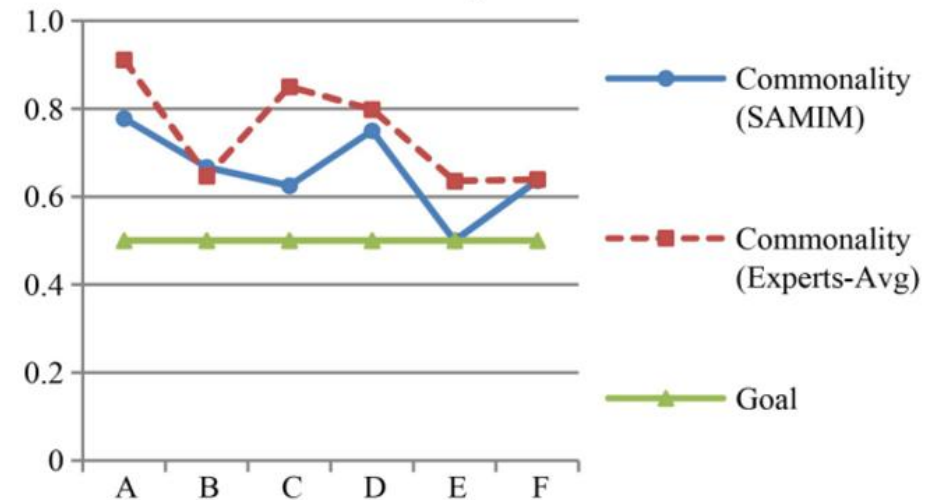


Figura 6. Comparação dos valores de comunalidade obtidos pelo SAMIM e pelos especialistas nos seis casos experimentais. Fonte: Khoshnevis [2].

MSDC

Busca otimizar uma decomposição **inicial** do monólito
em MS – MSDC

MSDC? Cromossomo? Solução candidata?

- **Entrada**

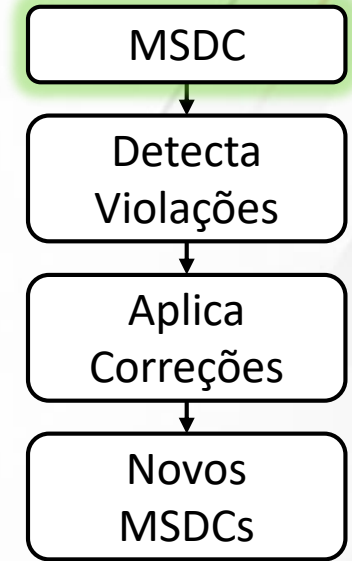
- **Lista de exceções de violações**

- Também definida pelo arquiteto;
 - Chamadas entre MS que o arquiteto considera aceitáveis;
 - Evita correções indesejadas

- **MSDC inicial** (*Criado pelo arquiteto de TI*)

- Decomposição inicial;
 - Baseada por funções de negócio.

- **MSDC? Qual formato? Como é gerado?**



- O estudo define um **metamodelo** que considera 12 elementos:

- API, API Gateway, API MS; API Call; API Group;
- Classes e Métodos;
- Banco de dados; Tabelas e Colunas;
- Relação entre métodos e CRUD.
- Chamadas entre métodos.

API Gateway “Clientes”

- API cadastrarCliente
- método cadastrarCliente()
- acessa tabela Cliente

- O estudo define o metamodelo do MSDC inicial, mas...

- **Mas não define como obtê-la.**

MS contém classes

Classe contém métodos

API chama um método

Métodos podem chamar outros métodos

Métodos acessam colunas via CRUD

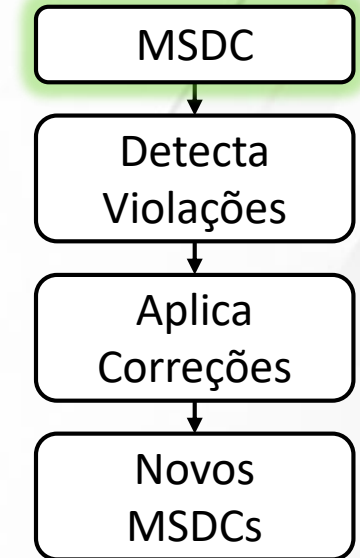
- O MSDC inicial precisa representar funções de negócio;
- Exemplo:

Monólito de banco

- função **Clientes** → MS Clientes
- função **Contas** → MS Contas
- função **Transferências** → MS Transferências

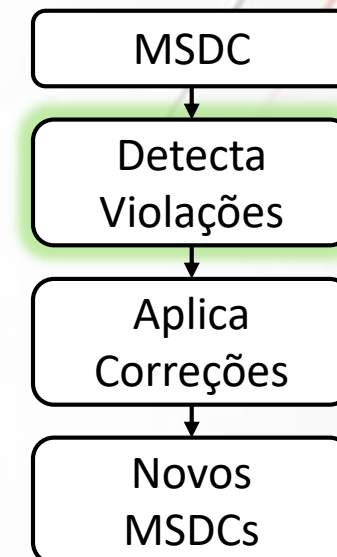
- Definida pelo arquiteto no MSDC inicial;
- São usadas para validar políticas.

- **Com a ENTRADA (MSDC inicial), o algoritmo segue a próxima fase: Detectar Violações... Como?**



Políticas de decomposição

- **P1 — Capacidade de negócio**
 - Um MS → uma capacidade de negócio
 - **P2 — Chamadas entre MS**
 - Somente chamadas previamente especificadas pelo arquiteto
 - **P3 — Acesso ao BD**
 - Cada tabela → escrita por um único MS
 - **P4 — Transações**
 - Evitar transações distribuídas entre MS
- **O algoritmo verifica violações do MSDC por meio destas políticas**



Violações e avaliação

- Para cada violação ele define a proporção de elementos envolvidos

- $P1 \rightarrow EF1$
- $P2 \rightarrow EF2$
- $P3 \rightarrow EF3$
- $P4 \rightarrow EF4$

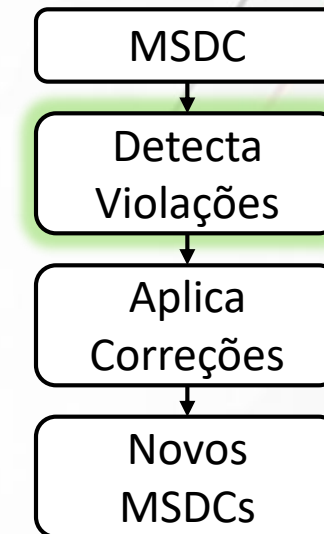
$$EF_n = \frac{\text{elementos do tipo em violação}}{\text{elementos considerados do tipo}}$$

- Gera o “**vetor de objetivos**” (Efs simultâneos);

$[EF1, EF2, EF3, EF4]$ – determina dominância (Pareto)

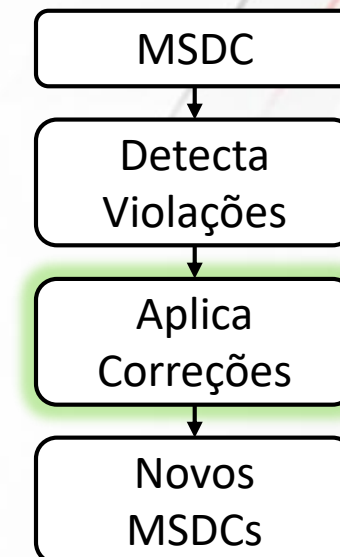
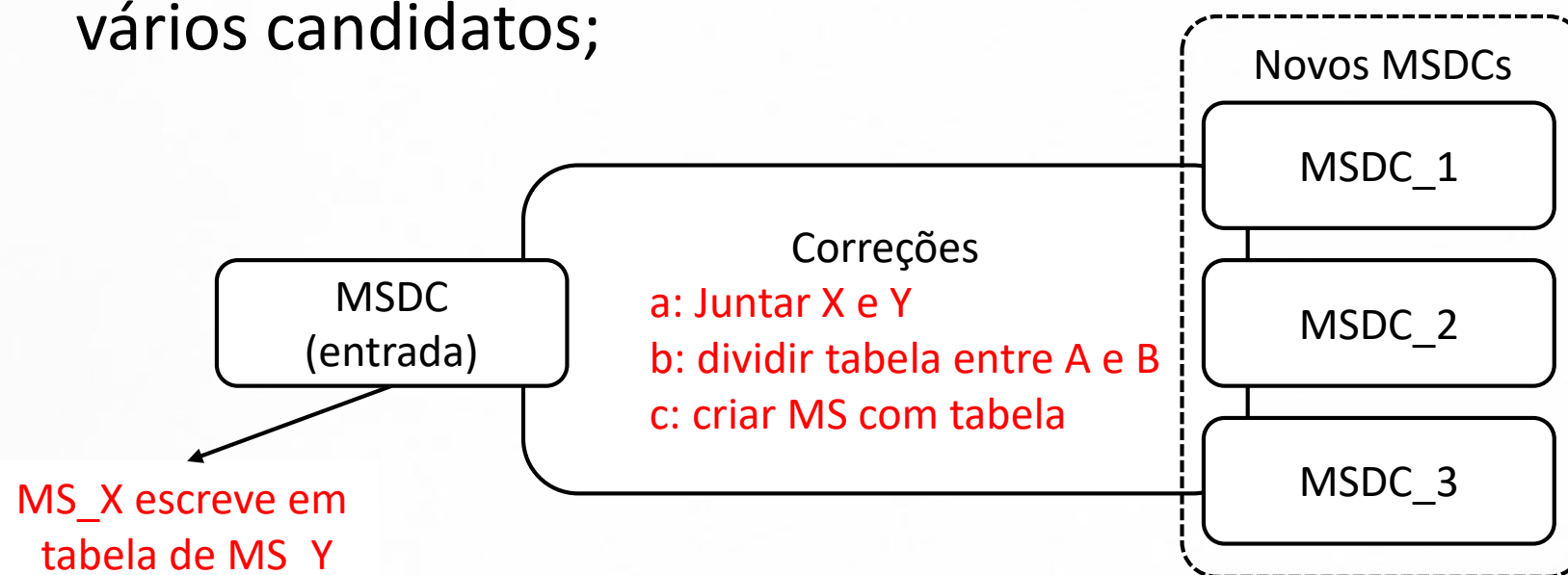
O método busca minimizar simultaneamente as quatro funções.

- Quanto menor o valor das quatro funções, menor a ocorrência das violações.



Correções

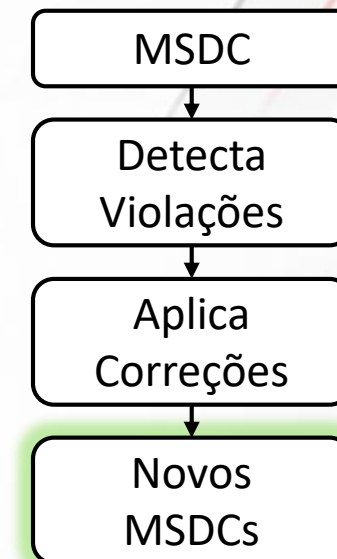
- Depois de detectar uma violação (V1–V4);
- Aplica a correção da violação específica;
- Cada alteração gera um novo MSDC candidato;
- A mesma violação pode ter várias correções e gerar vários candidatos;



Novos MSDCs

- **Novos MSDCs**

- Calcula **o vetor de objetivos** para cada candidato
- Ordenação não dominada (Pareto)
- **Pareto-ótimos** seguem para a **próxima iteração** (Violações, Correções, Novos)
- **Limitado a 100 iterações**
- Geração dos candidatos ocorre por operações de correção predefinidas, não usa operadores genéticos
 - não tem seleção genética, cruzamento (*crossover*) ou mutação



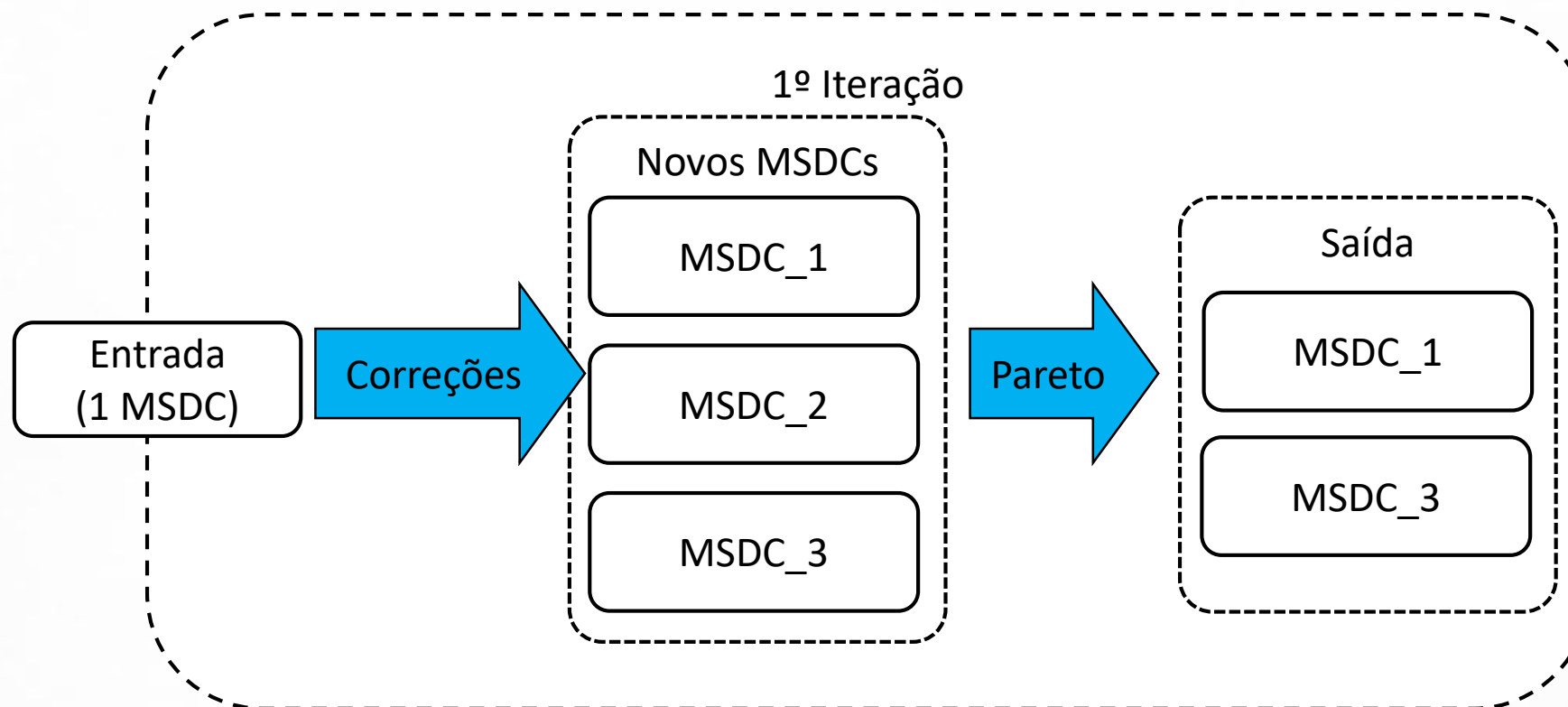


Figura 7. Exemplo da primeira iteração do processo de geração e seleção de MSDCs. Fonte: elaboração própria com base em Kinoshita e Kanuka [3].

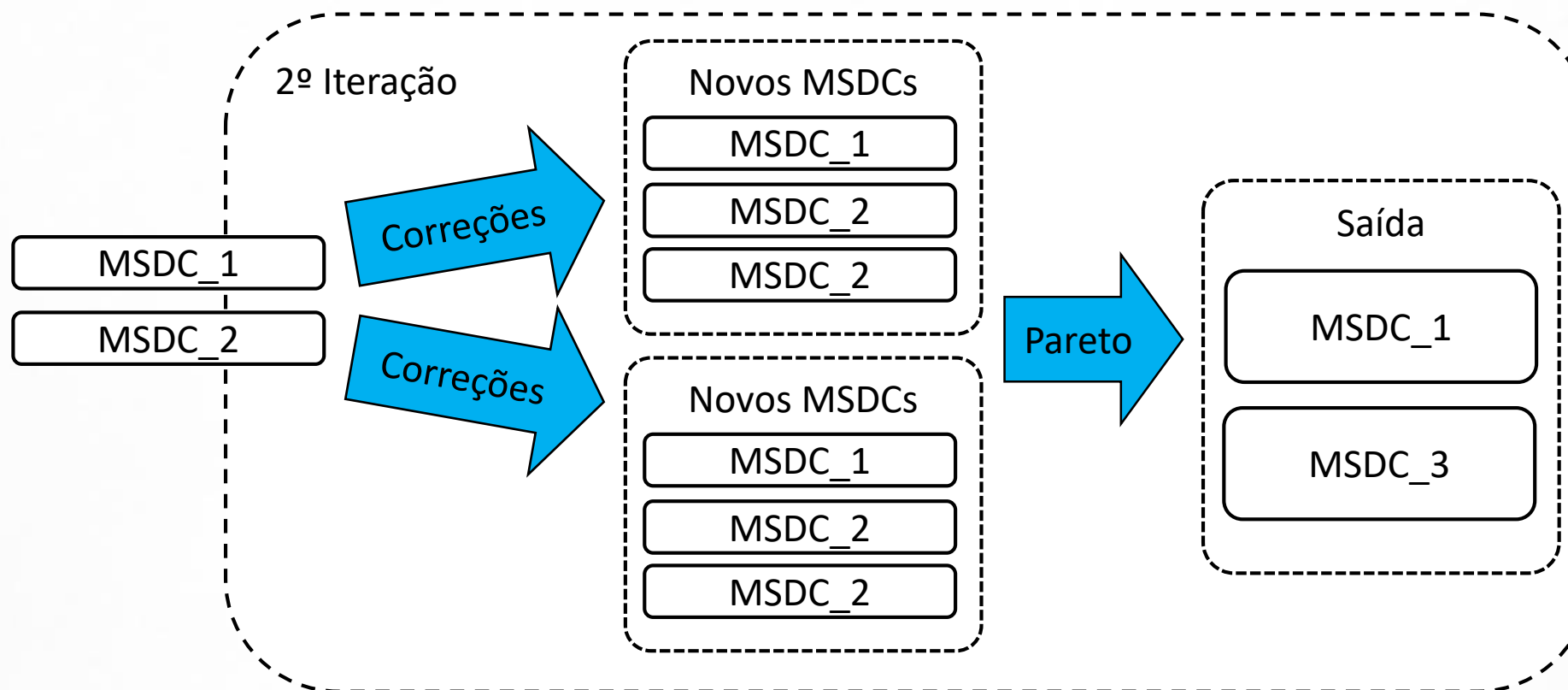


Figura 8. Exemplo da segunda iteração do processo de geração e seleção de MSDCs. Fonte: elaboração própria com base em Kinoshita e Kanuka [3].

Segunda etapa de seleção

- O conjunto de Pareto-ótimos pode conter muitas soluções;
 - Gera **problema de escolha**;
- Arquiteto define o número de soluções;
- Algoritmo aplica as **linhas de referência**
 - método gera vetores de referência distribuídos no espaço dos quatro objetivos
- Seleciona MSDCs representativos de diferentes regiões do Pareto
- Entrega o conjunto com a quantidade de soluções definida pelo arquiteto
- Arquiteto escolhe e, se necessário, refina

Experimento

3 aplicações de software

- PET CLINIC
 - ≈ 1 mil linhas de código
 - Open source · Spring Boot
 - Clínica veterinária
 - 4MSDCs
- INTERNET BANKING
 - ≈ 10 mil linhas de código
 - Struts · Spring · MySQL · iBatis
 - Desenvolvida para um banco financeiro
 - 220 MSDCs
- GERENCIAMENTO DE CONTRATOS DE SEGUROS
 - ≈ 20 mil linhas de código
 - Spring MVC · PostgreSQL · MyBatis
 - Desenvolvida para empresa de seguro de vida
 - 243 MSDCs

Método de avaliação

- Avaliação estruturada (5 itens → EI1–EI5)
- Análise da geração e diversidade das decomposições
- Análise do tempo de geração
- Análise da seleção das soluções conforme diferentes prioridades
- Verificação da aderência das soluções às prioridades definidas
- Seleção e refinamento por arquitetos de TI experientes

Principais resultados

- Prioridades definidas refletidas nas soluções selecionadas (EI3/EI4)
- Correlação positiva entre prioridades e características das soluções (EI4)
- Arquitetos utilizaram essas informações para priorizar políticas, selecionar e refinar a solução (EI5)

Fontes centrais

- Sellami, K. et al. Improving microservices extraction using evolutionary search. *Information and Software Technology*, 2022.
- Khoshnevis, S. A search-based identification of variable microservices for enterprise SaaS. *Frontiers of Computer Science*, 2023.
- Kinoshita, T.; Kanuka, H. Enhancing automated microservice decomposition via multi-objective optimization. *IEEE Access*, 2024.